WEEK 1 · DAY 1 · MONDAY 19 OCTOBER 2026

Tokenization

How text becomes numbers — and why it sets your bill

BPE · WordPiece · Unigram · SentencePiece · byte-level BPE

Special tokens & chat templates · quirks lab · vocabulary trade-offs

2026 addition: tokenizer differences & the non-English token tax (measured)

Week 1 · Foundations — LLM internals

Instructor: Pro AI Principal Architect

1. Why Tokenization Is Day 1

A language model never sees text. It sees a list of integers. **Tokenization** is the step that turns your string into those integers, and turns the model's integers back into text. Every number you will care about as an AI engineer — **cost, latency, context-window capacity, rate limits, and even accuracy** — is measured in tokens, not characters or words.

> **Architect's one-liner**
> The tokenizer is part of the model's contract. Change the model and you may change the tokenizer — and with it your bill, your context budget and your failure modes.

The pipeline

```
"Tokenization matters!"
       │  1. normalise      (Unicode NFC/NFKC, optional lower-casing)
       ▼
"Tokenization matters!"
       │  2. pre-tokenise   (split on spaces, punctuation, digit runs)
       ▼
["Token","ization"," matters","!"]   ◄— 3. subword model (BPE / WordPiece / Unigram)
       │  4. map to IDs     (lookup in the vocabulary)
       ▼
[ 3404, 2065, 13146, 0 ]   ◄— cl100k_base, measured
       │  5. embedding lookup → vectors → transformer   (Day 2, Day 3)
       ▼
model output IDs → detokenise → text
```

2. Three Ways to Cut Text

Granularity	Example: "unbelievable"	Strength	Fatal weakness
Word-level	[unbelievable]	Short sequences; intuitive	Huge vocabulary; any unseen word becomes [UNK] (out-of-vocabulary)
Character-level	[u,n,b,e,l,i,e,v,a,b,l,e]	Tiny vocabulary; never [UNK]	Very long sequences; attention cost grows with length; meaning is spread thin
Subword (modern LLMs)	[un, belie, vable] (cl100k_base, measured)	Frequent words stay whole; rare words decompose; no [UNK]	Splits are statistical, not linguistic — causes quirks (Section 6)

Subword tokenization is the compromise every frontier and open-weight LLM uses today. The vocabulary is a fixed set of pieces (typically ~30K to ~260K) learned from a training corpus.

3. Byte Pair Encoding (BPE) — The Workhorse

Training idea: start with single symbols, count every adjacent pair across the corpus, merge the most frequent pair into a new symbol, and repeat until the vocabulary reaches the target size. The ordered list of merges *is* the tokenizer. BPE was adapted from data compression to neural machine translation by Sennrich et al. (2016).

Encoding a new word: split into characters, then apply the learned merges in the same order.

Worked run — real output from week-01/notebooks/day01_bpe_from_scratch.py

Corpus (word: frequency) = low: 5, lower: 2, newest: 6, widest: 3. The marker </w> means end-of-word.

```
merge 1: 'e' + 's' -> 'es'  (count=9)
merge 2: 'es' + 't' -> 'est'  (count=9)
merge 3: 'est' + '</w>' -> 'est</w>'  (count=9)
merge 4: 'l' + 'o' -> 'lo'  (count=7)
merge 5: 'lo' + 'w' -> 'low'  (count=7)
merge 6: 'n' + 'e' -> 'ne'  (count=6)
merge 7: 'ne' + 'w' -> 'new'  (count=6)
merge 8: 'new' + 'est</w>' -> 'newest</w>'  (count=6)
merge 9: 'low' + '</w>' -> 'low</w>'  (count=5)
merge 10: 'w' + 'i' -> 'wi'  (count=3)
lowest -> ['low', 'est</w>']
newer -> ['new', 'e', 'r', '</w>']
```

```
wider -> ['wi', 'd', 'e', 'r', '</w>']
```

Notice: **lowest** never appeared in training, yet it encodes cleanly as *low* + *est*. **newer** and **wider** fall back to single characters for the parts the merges never covered. That graceful fallback is why subword models have no out-of-vocabulary problem.

Byte-level BPE — why modern models never emit [UNK]

Instead of starting from Unicode characters, byte-level BPE starts from the **256 possible byte values**. Any string in any language, emoji or binary-ish noise is a sequence of UTF-8 bytes, so it can always be encoded. GPT-2 popularised this; the open-source **tiktoken** library implements the same family (encodings *gpt2*, *cl100k_base*, *o200k_base*). Before merging, a regex **pre-tokeniser** splits text into chunks (words with their leading space, digit runs, punctuation) so merges never cross those boundaries.

4. The Other Algorithms and Who Uses Them

Algorithm	How it builds the vocabulary	Notes
BPE	Bottom-up: merge the most frequent adjacent pair	Deterministic, simple, fast; byte-level variant has no [UNK]
WordPiece	Bottom-up: merge the pair that most increases training-data likelihood (score ≈ count(ab) ÷ (count(a)·count(b)))	Google's BERT; continuation pieces are marked with ## (e.g. play, ##ing)
Unigram LM	Top-down: start with a huge candidate set, repeatedly prune pieces that least hurt likelihood	Probabilistic — a word can have several segmentations; enables subword regularisation (Kudo, 2018)
SentencePiece	A library/framework, not a separate algorithm — runs BPE or Unigram directly on raw text	Treats the space as a symbol (▁), so it is language-agnostic and fully reversible; from Google

FAANG tokenizer choices (published figures)

Company / model	Tokenizer	Vocabulary	Why it matters
Google — BERT	WordPiece	~30K	Classic encoder baseline; the ## convention you will see in NLP code
Google — T5	SentencePiece (Unigram)	32K	Text-to-text models; reversible ▁ spaces
Google — PaLM / Gemma	SentencePiece	~256K	Very large multilingual vocabulary reduces fragmentation in languages such as Tamil
Meta — Llama 2	SentencePiece (BPE)	32K	Small vocab → more tokens per sentence
Meta — Llama 3	tiktoken-based BPE	128K (100K + 28K non-English)	Meta reports English compression improved from 3.17 to 3.94 characters per token vs Llama 2
Apple — Apple Intelligence (2024 report)	BPE-family	49K on-device / 100K server	Smaller vocabulary on device saves memory in the embedding table (Section 7)

5. Special Tokens and Chat Templates

Every vocabulary reserves **special tokens** that never come from normal text: beginning/end of sequence, padding, and the markers that separate system, user and assistant turns. A **chat template** is the exact recipe for wrapping a conversation in those markers. Example (Meta Llama 3 family):

```
<|begin_of_text|><|start_header_id|>system<|end_header_id|>

You are a helpful assistant.<|eot_id|><|start_header_id|>user<|end_header_id|>

What is a token?<|eot_id|><|start_header_id|>assistant<|end_header_id|>
```

- **Wrong template = silent quality loss.** The model still answers, just worse. When self-hosting open models, always apply the template shipped with the tokenizer.
- **Security:** if user text containing the string "**<|endoftext|>**" were encoded as the real special token, a user could forge turn boundaries. tiktoken refuses by default — measured below.

```
>>> enc = tiktoken.get_encoding("cl100k_base")
>>> enc.encode("<|endoftext|>")
ValueError: Encountered text corresponding to disallowed special token '<|endoftext|>'.
>>> enc.encode("<|endoftext|>", allowed_special="all")   # trusted text only
[100257]
>>> enc.encode("<|endoftext|>", disallowed_special=())     # treat as plain text
[27, 91, 8862, 728, 428, 91, 29]
```

6. Quirks Lab — Real cl100k_base Output

Input	Token IDs	Pieces	Lesson
'hello'	[15339]	hello	Common word = 1 token
' hello'	[24748]	␣hello	Leading space creates a **different** token
'Hello'	[9906]	Hello	Case creates a different token
' HELLO'	[38757, 1623]	␣HEL | LO	Rare casing fragments
'strawberry'	[496, 675, 15717]	str | aw | berry	Why letter-counting is hard: the model never sees individual letters
' strawberry'	[73700]	␣strawberry	Same word, one token with a leading space
'1234567'	[4513, 10961, 22]	123 | 456 | 7	Numbers are chunked in up to 3 digits — arithmetic sees chunks, not digits
'2026-10-12'	[2366, 21, 12, 605, 12, 717]	202 | 6 | - | 10 | - | 12	Dates cost 6 tokens; the year is split
' tokenization'	[4037, 2065]	␣token | ization	Morphology emerges from statistics

Production lesson — the prompt boundary problem
Because ' hello' and 'hello' are different tokens, a prompt that ends with a trailing space pushes the model into an unusual token path and can degrade the completion. **Microsoft**'s open-source Guidance library addresses this with *token healing*: it backs up one token at the prompt boundary and constrains generation to continue the removed text. Practical rule: strip trailing whitespace from prompts.

7. Tokens Are Money, Latency and Memory

- **Money.** Providers price input and output per token (e.g. **Amazon** Bedrock on-demand pricing). Output tokens usually cost more than input tokens.
- **Latency.** Input tokens are processed in parallel; output tokens are generated one at a time. Output length dominates response time.
- **Context.** A "128K context window" means 128K *tokens of that model's tokenizer* — not a fixed amount of text.
- **Rule of thumb (English only):** roughly 4 characters or ~0.75 words per token. Never apply it to other languages or code — measure.

The vocabulary-size trade-off

The embedding table holds one vector per token: **parameters = V × d**. For a 128,256-token vocabulary and a 4,096-dimensional model (Meta Llama 3 8B sizes) that is 128,256 × 4,096 ≈ **525 million parameters**, about 1.05 GB in 16-bit precision — before the output layer, which has a matching matrix unless weights are tied.

Bigger vocabulary	Smaller vocabulary
Fewer tokens per sentence → cheaper, faster, more text fits in context	More tokens per sentence → higher cost and latency
Better coverage of many languages and code	Rare languages fragment into bytes
Larger embedding/output matrices → more memory, bigger softmax	Smaller memory footprint — why **Apple** chose 49K on-device vs 100K on server
Rare tokens get little training signal (under-trained 'glitch' tokens)	Each token is seen more often in training

8. 2026 Addition — Tokenizer Differences and the Token Tax

The same sentence costs different numbers of tokens on different model families, and non-English text usually costs **far** more. Measured with week-01/notebooks/day01_token_tax.py:

English: *Artificial intelligence is changing the world.* (46 characters)

Tamil: செயற்கை நுண்ணறிவு உலகை மாற்றுகிறது. (35 characters, 97 UTF-8 bytes — same meaning)

Encoding	Vocabulary	English tokens	Tamil tokens	Tamil token tax
gpt2	50,257	8	97	**12.1×**
cl100k_base	100,277	8	49	**6.1×**
o200k_base	200,019	7	14	**2.0×**

- **Why 97 tokens for gpt2?** Each Tamil character is 3 bytes in UTF-8. With almost no Tamil merges learned, byte-level BPE falls back to one token per byte: 97 bytes → 97 tokens.
- **Why it improves:** larger, deliberately multilingual vocabularies learn Tamil merges. Meta added 28K non-English tokens to Llama 3; Google's ~256K SentencePiece vocabulary was designed for broad language coverage.
- **Research backing:** Petrov et al., *Language Model Tokenizers Introduce Unfairness Between Languages* (NeurIPS 2023), found up to 15× differences in tokenized length for the same content across languages.

What the token tax does to a product

Effect	Impact for a Tamil-speaking user (6.1× tax example)
Cost	The same answer costs ~6× more
Context	A fixed token window holds ~1/6 as much Tamil text — RAG can retrieve fewer documents
Latency	Generating a Tamil reply takes many more decode steps
Quality	Fragmented tokens carry less meaning each; models are often weaker in heavily fragmented languages
Rate limits	Tokens-per-minute quotas are exhausted sooner

Cross-family rules for 2026

1. **Compare cost per task, not price per token.** A cheaper per-token model with a less efficient tokenizer for your language can cost more per answer.
2. **Count with the model's own tokenizer.** Use the provider's token-counting API or the published tokenizer — never another family's tokenizer as a proxy.
3. **Budget per language.** Set context, cost and rate-limit budgets per locale, measured on real traffic.
4. **Re-measure on every model migration.** Model churn in 2026 is monthly; a tokenizer change silently moves every cost and capacity number.

9. Real-World Scenarios

Company	Scenario	Tokenization decision
Meta	Llama 3 serves many languages to open-weight users	Grew vocabulary 32K → 128K, adding 28K non-English tokens; better compression with no English regression (Meta's paper)
Google	Gemma / PaLM must work across many scripts	~256K SentencePiece vocabulary to avoid fragmenting languages such as Tamil
Apple	On-device model with tight memory	49K vocabulary on device vs 100K on server (2024 report) — vocabulary size is a memory decision
Microsoft	Prompt-boundary artefacts in constrained generation	Guidance library's token healing
Amazon	Bedrock bills each model per input/output token	Customers must forecast spend with each model's own tokenizer
Netflix *(illustrative scenario)*	Generating synopses in many languages	A per-language token budget would be needed: a fixed English budget would truncate or overspend in fragmented languages

10. Architect's Lens

Decision	Question to ask	Default
Model choice	What is the token tax for *our* languages and data (code, JSON, tables)?	Measure on a 1,000-sample real-traffic set before choosing
Capacity planning	Tokens/sec and tokens/minute at p95 — per locale?	Plan in tokens, never characters
Context budget	How many tokens are left for retrieval after system prompt + history?	Reserve output tokens first
Self-hosting	Is the exact tokenizer + chat template shipped with the weights?	Pin tokenizer version with model version
Security	Can user input be encoded as special tokens?	Encode user text as plain text only
Migration	Did the tokenizer change between model versions?	Re-run cost and context tests on every upgrade

Failure modes to recognise in production

- Bill jumps after a model upgrade with no traffic change → tokenizer changed.
- Non-English users report truncated answers → context budget set in English token terms.
- Model miscounts letters or digits → it sees chunks, not characters; use code/tools for exact counting.
- Self-hosted model answers poorly → wrong chat template.
- Weird completions after prompts ending in spaces → prompt-boundary problem.

11. Hands-On Exercises

1. **Run BPE from scratch.** Execute week-01/notebooks/day01_bpe_from_scratch.py. Change the corpus to {"hug": 10, "pug": 5, "pun": 12, "bun": 4, "hugs": 5} and predict the first 3 merges before running.
2. **Measure your own token tax.** Add a Tamil paragraph from a news site and its English translation to day01_token_tax.py. Record the tax for all three encodings.
3. **Break the model.** Encode 'strawberry', ' strawberry', 'Strawberry' and 'STRAWBERRY' with cl100k_base. Explain why letter-counting questions fail.
4. **Number chunks.** Encode 1000000, 1,000,000 and 1e6. Which format would you choose in prompts that ask for arithmetic, and why?
5. **Architect write-up (½ page).** Your product serves English and Tamil users. Using today's numbers, write how you would set the context budget and monthly cost forecast per language.

```
pip install tiktoken
python week-01/notebooks/day01_bpe_from_scratch.py
python week-01/notebooks/day01_token_tax.py
```

12. Key Takeaways

1. Models read integer token IDs; the tokenizer is part of the model's contract.
2. Subword tokenization (BPE, WordPiece, Unigram via SentencePiece) balances vocabulary size and sequence length.
3. Byte-level BPE starts from 256 bytes, so nothing is ever out-of-vocabulary.
4. Cost, latency, context and rate limits are all measured in tokens.
5. Spaces, case and digits change tokenization — the root of many 'dumb' model failures.
6. Vocabulary size trades memory against sequence length.
7. 2026: the same text costs different token counts per model family, and non-English text can cost 2–12× more (measured) — compare cost per task, measure per language, and re-measure on every model migration.

Next — Day 2 · Tue 20 Oct 2026 · Embeddings
Token IDs become vectors. 2026 addition: Matryoshka (truncatable) embeddings, multi-vector late interaction (ColBERT) and multimodal embeddings.

References

- Sennrich, Haddow & Birch (2016), *Neural Machine Translation of Rare Words with Subword Units*.
- Kudo (2018), *Subword Regularization*; Kudo & Richardson (2018), *SentencePiece*.
- Petrov et al. (2023), *Language Model Tokenizers Introduce Unfairness Between Languages*, NeurIPS.
- Meta (2024), *The Llama 3 Herd of Models* — arxiv.org/abs/2407.21783.
- Apple (2024), *Apple Intelligence Foundation Language Models* — machinelearning.apple.com.
- Microsoft Guidance — github.com/guidance-ai/guidance (token healing).
- tiktoken — github.com/openai/tiktoken.